

Day 3: Parameterization of Monomers and Polymer Chain Generation

This tutorial demonstrates how to parameterize monomers and generate polymer chains using **AmberTools**. It includes reusable functions to simplify the process for new molecules and provides visualization tools for both monomers and polymers.

Table of Contents

1. [Initialization](#)
 2. [Prepare Monomer](#)
 3. [Parameterize Monomer](#)
 4. [Define Chain, Head, and Tail](#)
 5. [Generate Polymer Chain](#)
 6. [Convert to GROMACS Formats](#)
 7. [Visualize Monomer and Polymer](#)
-

Initialization

The first step is to set up the environment for the tutorial. This includes:

- Opening the terminal and navigating to the `polymermd-workshop-main` folder:

```
cd /PATH_TO/polymermd-workshop-main
```

Replace `PATH_TO` to the directory path in your computer.

- Activating the conda environment:

```
conda activate polymer-md
```

- Launching Jupyter Lab:

```
jupyter lab --ip 0.0.0.0 --no-browser
```

Open the printed URL in your browser, or configure it to open automatically.

- Installing required Python packages (`nglview`, `parmed`).
- Setting up the project root directory.

- Creating subdirectories for monomers and polymers.
- Defining reusable functions for running commands, managing directories, and visualization.

Run the initialization cell to set up the environment.

Prepare Monomer

This step converts a SMILES string, PDB, or XYZ file into a 3D molecular structure in MOL2 format.

AmberTool Used: antechamber

Parameters:

- **mol_name**: Name of the molecule.
- **input_type**: Type of input file (**smiles**, **pdb**, or **xyz**).
- **input_data**: SMILES string or path to the input file.

Example:

```
prepare_monomer(mol_name="PEO", input_type="smiles",  
input_data="CCOCCOCCOCCOCCOCC")
```

Parameterize Monomer

This step uses **antechamber** to assign GAFF atom types and AM1-BCC charges to the monomer.

AmberTool Used: antechamber

Parameters:

- **mol_name**: Name of the molecule.

Example:

```
parameterize_monomer(mol_name="PEO")
```

Define Chain, Head, and Tail

This step defines the chain, head, and tail of the monomer for polymerization. It also generates the required **.prepi** files using **prepgen**.

AmberTool Used: prepgen

Parameters:

- `mol_name`: Name of the molecule.
- `head_id`, `tail_id`: Atom indices for the head and tail.
- `head_omit`, `tail_omit`: Atom indices to omit near the head and tail.

Example:

```
define_chain_head_tail(  
    mol_name="PEO",  
    head_id="C1",  
    tail_id="C10",  
    head_omit=["C", "H", "H1", "H2"],  
    tail_omit=["C11", "H23", "H24", "H25"]  
)
```

Generate Polymer Chain

This step builds a polymer chain using the defined monomer, head, and tail. The `generate_polymer_tleap` function is used to generate homopolymers, block copolymers, or random copolymers.

AmberTool Used: `tleap`

Function:

```
generate_polymer_tleap(  
    mol_names,          # List of monomer names  
    n_mono_repeat,      # Number of monomers per repeat unit  
    n_mono_pol,         # Total number of monomers in the polymer  
    copolymer_type,     # "homo", "block", or "random"  
    head_group=None,    # Head capping group (default: first monomer)  
    tail_group=None     # Tail capping group (default: last monomer)  
)
```

Examples:

Homopolymer

Generate a homopolymer of PEO with 25 monomers:

```
generate_polymer_tleap(  
    mol_names=["PEO"],  
    n_mono_repeat=[5],  
    n_mono_pol=[25],  
    copolymer_type="homo",  
    head_group="PEO",
```

```
    tail_group="PE0"  
)
```

Block Copolymer

Generate a block copolymer of PEO (25 monomers) and PET (25 monomers):

```
generate_polymer_tleap(  
    mol_names=["PE0", "PET"],  
    n_mono_repeat=[5, 1],  
    n_mono_pol=[25, 25],  
    copolymer_type="block",  
    head_group="PE0",  
    tail_group="PET"  
)
```

Random Copolymer

Generate a random copolymer of PEO (25 monomers) and PET (25 monomers):

```
generate_polymer_tleap(  
    mol_names=["PE0", "PET"],  
    n_mono_repeat=[5, 1],  
    n_mono_pol=[25, 25],  
    copolymer_type="random",  
    head_group="PE0",  
    tail_group="PET"  
)
```

The polymer chain is generated using `tleap`, and the output files include:

- PDB file: `polymer_<N>mer.pdb`
- Topology file: `polymer_<N>mer.prmtop`
- Coordinate file: `polymer_<N>mer.inpcrd`

Convert to GROMACS Formats

This step converts the AMBER topology and coordinate files into GROMACS-compatible formats using `parmed`.

AmberTool Used: `parmed`

Example:

```
amber = pmd.load_file(f"{polymer_dir}/polymer_{n_mono_pol}mer.prmtop", f"{polymer_dir}/polymer_{n_mono_pol}mer.inpcrd")
amber.save(f"{polymer_dir}/polymer_{n_mono_pol}mer.gro", overwrite=True)
amber.save(f"{polymer_dir}/topol.top", format="gromacs", overwrite=True)
```

Visualize Monomer and Polymer

Visualize Monomer

The monomer can be visualized using its MOL2 file. Atom indices are displayed, and the camera is set to **orthographic**.

Example:

```
visualize_monomer("PEO")
```

Visualize Polymer

The polymer can be visualized using its PDB or GROMACS **.gro** file. Atom indices are displayed, and the camera is set to **orthographic**.

Example:

```
visualize_molecule(f"{polymer_dir}/polymer_{n_mono_pol}mer.gro")
```

Notes

- Ensure that all required dependencies are installed before running the notebook.
- The **PROJECT_ROOT** directory is set dynamically and used to organize monomer and polymer files.
- The visualization functions use **nglview** for interactive 3D visualization.

License

This tutorial is provided under the MIT License. Use it freely for educational and research purposes.